GUIDE 55 · OPERATE

Manage your setup as code, and promote it from dev to prod through a pull request

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=55-promote-a-setup-from-dev-to-prod-through-a-pull-request. This copy was made on 4 November 2026.

Time: 20 minutes. **You'll end up with:** dev's connections, queries, roles and keys in a file in git; prod made to match it by CI after a reviewed pull request, all or nothing; and prod refusing changes made any other way.

> **Stable** - configuration as code, its file's `format_version` 1 and the commands, are under the compatibility promise since 0.30 (what that means).

Every command and output below was run against two real servers: **dev** and **prod**, each with its own store and its own SQLite database of sales, with prod started with `QUERYAPIGATE_CONFIG_LOCKED=1`. The pull request was a branch in a real git repository; CI's steps were run by hand.

Step 1: Dev, as you'd build it

On dev, a connection, a saved query in the `finance` collection, and a role for partners - made in the Console or the API as usual. The connection's database is a **reference**, so the same file works where the path differs:

```json
POST /api/v1/connections
{"name": "sales", "db": "sqlite", "database": "${SALES_DB}"}
```

Each server sets `SALES_DB` to its own database. (A password is a reference the same way - `${WAREHOUSE_PASSWORD}` - and a file refuses one that isn't.)

Step 2: Dev's setup, as a file in git

```shell
QUERYAPIGATE_HOME=/srv/dev queryapigate config export --out queryapigate.yaml
```

```yaml
format: queryapigate-config
format_version: 1
connections:
- name: sales
  db: sqlite
  active: true
  database: ${SALES_DB}
roles:
- name: partner
  connections: []
  collections:
  - finance
  rate_limit: 60/minute
queries:
- name: monthly_revenue
  collection: finance
  connection_name: sales
  description: Revenue per month
  author: admin
  sql: SELECT month, amount FROM revenue ORDER BY month
api_keys: []
```

Add the partner's key. A key is in a file by its secret's **hash**, never the secret:

```shell
queryapigate config new-key
```

```text
secret:        sk_pdiK5...      # to the partner, and your secrets manager - not the file
secret_sha256: 3cf3a4f9023214f61d2d657c95c1e349765d70819994ec940ebb3db012da4e0d
```

```yaml
api_keys:
- name: partner-a
  role: partner          # the role's grants - kept in step with it
  secret_sha256: 3cf3a4f9023214f61d2d657c95c1e349765d70819994ec940ebb3db012da4e0d
```

Commit it. Nothing in the file is a secret, so it can live in the same repository as everything else.

Step 3: Prod, from the file

What CI runs against prod - the CLI posts the file to the server and shows its answer; the key comes from `QAG_API_KEY`, an owner's (only owners can apply a file):

```shell
export QAG_API_KEY=<prod owner key>    # a CI secret
queryapigate config diff  --server https://prod.example.com queryapigate.yaml
queryapigate config apply --server https://prod.example.com queryapigate.yaml
```

The diff, on an empty prod (each object's fields left out here):

```text
+ connections[sales]
+ queries[monthly_revenue]
+ roles[partner]
+ api_keys[partner-a]

4 to create, 0 to change.
```

Applied - in one transaction, each change through the same rules as the API:

```text
Applied: the server matches the file (sha256 7f25ab941ccd).
```

The partner's secret - which prod never saw, only its hash - works, on prod's own data:

```text
GET /q/monthly_revenue   (X-API-Key: sk_pdiK5...)
[{"month":"2026-08","amount":98000},{"month":"2026-09","amount":104500},{"month":"2026-
10","amount":51200}]
```

Step 4: A change, as a pull request

On a branch: the partner gets 120 calls a minute, and a new query.

```diff
@@ roles:
-  rate_limit: 60/minute
+  rate_limit: 120/minute
@@ queries:
+- name: best_month
+  collection: finance
+  connection_name: sales
+  description: The month with the most revenue
+  sql: |-
+    SELECT month, amount
+    FROM revenue
+    ORDER BY amount DESC
+    LIMIT 1
```

CI posts what it would do to prod on the pull request - the review is of this, not only of the YAML:

```shell
$ queryapigate config diff --server https://prod.example.com queryapigate.yaml
+ queries[best_month]
    collection: "finance"
    connection_name: "sales"
    description: "The month with the most revenue"
    query_type: "sql"
    sql:
      SELECT month, amount
      FROM revenue
      ORDER BY amount DESC
      LIMIT 1
~ roles[partner]
    rate_limit: "60/minute" → "120/minute"
~ api_keys[partner-a]
    rate_limit: "60/minute" → "120/minute"

1 to create, 2 to change.
```

(exit 2: there are changes.) The key changes too: its grants are the role's. A file with a problem fails this step with every problem listed, and nothing reaches prod.

Meanwhile, someone tries the shortcut - the same change straight on prod:

```text
PATCH /api/v1/roles/partner {"rate_limit": "120/minute"}
409 {"code": "config_managed", "error": "roles[partner] is managed by a configuration file - change it
there; this
     server takes changes to it only from a file (QUERYAPIGATE_CONFIG_LOCKED)"}
```

Step 5: Merged, applied

After the merge, CI applies it, then checks nothing is left over:

```shell
$ queryapigate config apply --server https://prod.example.com queryapigate.yaml
1 to create, 2 to change.
Applied: the server matches the file (sha256 a30e8ac0451d).

$ queryapigate config diff --server https://prod.example.com queryapigate.yaml
No changes: the server matches the file.
```

Prod's audit log says who applied which file, and each change it made:

```text
2026-10-12 16:31:22  admin  break-glass  config_applied  a30e8ac0451d
2026-10-12 16:31:22  admin  config       update_key      partner-a
2026-10-12 16:31:22  admin  config       update_role     partner
2026-10-12 16:31:22  admin  config       save_query      best_month
2026-10-12 16:30:33  admin  break-glass  config_applied  7f25ab941ccd
...
```

The hash names the file's content - the same file applied anywhere has the same one - so it ties each change to the commit that holds that file.

Step 6: Removing something

Delete `best_month` from the file and the diff, by default, leaves it alone - it only says so:

```text
On the server but not in the file - applying leaves them alone:
    queries[best_month]
```

With `--prune`, it's deleted - because a file put it there:

```shell
$ queryapigate config diff --prune --server https://prod.example.com queryapigate.yaml
- queries[best_month]

0 to create, 0 to change, 1 to delete.
```

`config apply --prune` then deletes it. Pruning only ever deletes what a file manages - created, changed or named by one - never what was made in the Console or the API and no file has named.

Good to know

- **A CI workflow** is these two steps: `config diff --server` on every pull request (post its output as a comment; exit 1 fails the check), and `config apply --server` on merge. The diff is read-only, but posts the file to the server: give both the owner key only as a protected secret.
- **On Kubernetes**, skip the CI apply: the chart's `configAsCode` mounts the file and the first new pod applies it as it starts (Install on Kubernetes with Helm), so Argo CD or Flux syncing the chart is the deploy. Elsewhere, `QUERYAPIGATE_CONFIG_FILE` does the same.
- **Without the lock**, a change made on prod shows up as drift: `config diff` names it, and the next apply undoes it. Run the diff on a schedule to see it.
- **Rotating the partner's key** is a pull request too: a new `secret_sha256` from `config new-key`. The old secret keeps working for 60 minutes, as a rotation does.
- **Use `config new-key`** for every key's secret. A hash is safe to keep in git only because the secret behind it is 32 random bytes; a secret someone chose could be guessed from its hash.

Next steps

- Configuration as code, in the reference.
- Rotate an API key without downtime.
- Read the threat model.